



算法综合实习课程报告

姓 名:	林佳欣	学 号:	20221001084
学 院:	计算机学院	专 业:	计算机科学与技术专业
选题名称:	卫星对地覆盖计算及任务规划		
指导教师:	陈晓宇		

2024 年 6 月

目 录

1 选题介绍..... 1

2 选题分析 3

 2.1 问题分析..... 3

 2.2 优化模型..... 4

 2.3 问题难点分析..... 5

3 算法设计与实现..... 7

 3.1 算法主要思想..... 7

 3.2 核心算法设计..... 8

 3.2.1 读取卫星文件算法思想 8

 3.2.2 合并时间窗口算法思想 8

 3.2.3 计算覆盖率算法思想 9

 3.2.4 贪婪任务调度算法思想 9

 3.3 数据结构设计..... 10

 3.4 算法模块设计..... 12

4 结果测试..... 16

 4.1 运行环境描述..... 16

 4.2 测试数据设计..... 16

 4.3 测试结果分析..... 17

 4.4 测试结果展示..... 18

5 分析总结..... 22

 5.1 课设总结..... 22

 5.2 特色亮点..... 22

 5.3 后期改进之处..... 22

6 附录..... 24

1 选题介绍

考虑一个场景：空间中，有多颗卫星绕地球运行，每颗卫星上都携带有一个传感器，其服务范围可以近似成一个圆锥形的区域。该锥形区域在地面上的投影为球面圆形的（注：是球面圆，不是平面圆）。如果该时刻，地面目标在该圆形区域内，则卫星可以对该地面目标提供服务，称作“地面目标在该时刻被卫星覆盖”。由于卫星在空间高速运行，使得卫星在不同时刻的对地服务区域是不断变化的。在本次实习中，我们主要从三个不同的角度分析卫星对多个地面目标覆盖服务的性能。一是卫星对目标的可见时间窗口的计算，二是卫星星座对区域的覆盖率计算，三是卫星星座对点目标观测任务的调度规划。

卫星对目标可见窗口的计算包括卫星星座对每个点目标的可见时间窗口，以及对每个点目标的二重覆盖时间窗口，还包括计算卫星星座对每个点目标的覆盖时间间隙，并统计每个点目标时间间隙的最大值和平均值，并将结果用以可视化形式展示出来。

卫星星座对区域的覆盖率计算是对一个经纬度矩形范围（经度区间为 75°E - 135°E ，纬度区间为 $(0^{\circ}\text{N}$ - $55^{\circ}\text{N})$ ）进行计算。包括对每个时刻瞬时覆盖率的计算，并将结果绘制成曲线。另一方面，对于仿真周期内的某个时刻，将该时刻区域内被覆盖的网格，不被覆盖的网格，不确定的网格用不同的颜色绘制出来。能够将不同时刻的覆盖率结果，以动态形式展现出来（即添加时间，能够对时间进行调整，让时间流动）结果大致呈现如下效果：

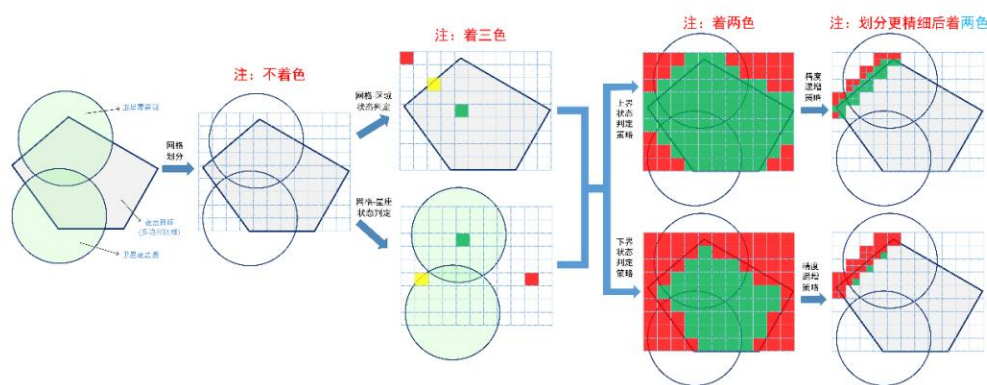


图 1-1 可视化效果示例

卫星星座对点目标观测任务的调度规划包括了五组测试算例，算法需要给出一个“最优”调度方案，最优调度方案应满足以下条件：（1）每个观测目标只能在自己的可见时间窗口内执行，否则，认为该目标未安排；（2）每个观测目标只能占用满足其

要求的资源集合中的一个资源，且执行过程不能中断；（3）每个卫星资源在任何时候只能同时满足一个目标的观测需求；（4）所有被安排观测目标的总权值最大。

五组测试算例如下：

仿真周期： [2024-01-01 11:00:00, 2024-01-01 12:30:00]			
卫星文件	Sat0, Sat6	目标文件	target1.txt
仿真周期： [2024-01-01 00:00:00, 2024-01-01 06:00:00]			
卫星文件	Sat0, Sat1, Sat2	目标文件	target2.txt
仿真周期： [2024-01-01 11:00:00, 2024-01-01 15:00:00]			
卫星文件	Sat0, Sat1, Sat3, Sat4, Sat5	目标文件	target3.txt
仿真周期： [2024-01-01 16:00:00, 2024-01-01 19:00:00]			
卫星文件	Sat3, Sat4, Sat5, Sat6, Sat7, Sat8	目标文件	target4.txt
仿真周期： [2024-01-01 00:00:00, 2024-01-01 12:00:00]			
卫星文件	Sat0, Sat1, Sat2, ..., Sat6, Sat7, Sat8	目标文件	target5.txt

表 1-1 测试算例

在任务规划的过程中，同一个卫星上连续的两个观测目标之间需要满足最小转换时长约束。假设卫星 Sat0 上观测任务的最小转换时长为 30 秒，则表明卫星 Sat0 在观测完一个目标后至少需要等待 30 秒后才可以观测下一个目标，而在这 30 秒内卫星不可以用来做其他的事情。所有卫星的转换时长约束如下表所示。

卫星资源	Sat0	Sat1	Sat2	Sat3	Sat4	Sat5	Sat6	Sat7	Sat8
任务间转换时长	30 秒	30 秒	30 秒	35 秒	35 秒	35 秒	25 秒	25 秒	25 秒

表 1-2 卫星的转换时长约束

要求完成如下操作：

- （1）使用贪心法，设计卫星星座对观测目标的调度规划算法。
- （2）使用动态规划或者任意一种智能优化算法实现该调度规划。

2 选题分析

在本次算法实习中，我们主要从三个不同的角度分析卫星对多个地面目标覆盖服务的性能，分别从点、面、时空三个方面对卫星性能进行分析。

2.1 问题分析

2.2.1 卫星对目标的可见时间窗口，即星座中每颗卫星对地面点目标可见时间窗口的并集。卫星服务范围可近似为一个圆锥形区域，该锥形区域在地面上的投影为球面圆形的。对于一个地面点目标，当目标在该圆形区域内时称作“地面目标在该时刻被卫星覆盖”，故可以将问题转换为目标点在圆形区域内的时间集合的并集。对于一个地面点目标，在某个时刻存在多颗卫星对其进行覆盖，则称点目标在该时刻被多重覆盖。点目标被多重覆盖的时间段集合，称作多重覆盖时间窗口。要求计算每个点目标的二重覆盖时间窗口即求每两个卫星对一个目标的时间窗口的交集。而卫星星座对每个点目标的覆盖时间间隙指的是目标点不被卫星星座覆盖的时间窗口，要求统计每个点的时间间隙的最大值和平均值可以转化为对目标所有窗口先求并集，再求对一天的补集。要求计算卫星星座对每个点目标的可见时间窗口，以及对每个点目标的二重覆盖时间窗口，将结果用以可视化形式展示出来。

2.2.2 在给定经纬度区域范围内，卫星对区域的覆盖率，等于被覆盖的面积与区域总面积之比。要直接计算区域目标被卫星群的覆盖面积是较为困难的，我们可以将区域目标划分为若干个小的网格，通过计算每个网格是否被卫星覆盖，从而得到近似的瞬时覆盖率计算结果。当网格面积远小于卫星覆盖范围时，只需要判断网格的四个顶点是否被卫星覆盖即可。如果四个顶点都被卫星覆盖，则该网格被卫星覆盖。如果四个顶点都没有被卫星覆盖，则网格没有被卫星覆盖。如果四个顶点有部分被卫星覆盖，有部分没有被卫星覆盖，则网格状态不确定。对于不确定的网格，我们可以将网格一分为四，然后判断新的网格是否会被卫星覆盖，直到不确定的网格面积之和与总面积之比小于 0.1% 停止。注意，完全覆盖或者完全没有覆盖的网格，不需要继续判断。该过程是一个分治的过程。通过上述过程，我们可以得到在任意时刻，卫星星座对目标的瞬时覆盖面积与区域总的面积之比，即星座对目标的瞬时覆盖率。

2.2.3 卫星星座对点目标观测任务的调度规划可以描述为在 m 个互不相同的卫星上（资源集合 R ）安排 n 个观测任务（点目标集合 M ）。对每个观测目标 $M_i \in M$ ，只有部分卫星可以满足其执行要求，因此，需要为尽可能多的目标选择某一个观测资源 R_j

$\in R$ ，以及一个对应的执行时间段 $[Beg_i, End_i]$ 。此外，观测目标 M_i 在占用资源 R_j 时具有一组互不相交的时间窗口集约束，且只能在其中的一个时间窗口内不中断地执行完成。如果目标 M_i 和目标 M_i' 在执行过程中占用同一个资源 R_j ，且目标 M_i 在目标 M_i' 之前执行，那么目标 M_i 执行完成后，必须经过一个转换时间 Δt ，观测 M_i' 才能开始执行。每个目标 M_i 都有权值 w_i ，代表该观测目标安排时的效益值。由于资源能力及时间的限制，并不是所有的观测目标都能够被安排。

2.2 优化模型

2.2.1 输入输出分析

输入为资源集合 $R = \{R_1, R_2, \dots, R_m\}$ 与任务集合 $M = \{M_1, M_2, \dots, M_n\}$ 。可见时间窗口集合为 $TW_{ij} = \{tw_{i,j}^1, \dots, tw_{i,j}^{N_{i,j}}\}$ ， $tw_{i,j}^k = [Beg_{i,j}^k, End_{i,j}^k]$ ，其中 $tw_{i,j}^k$ 表示任务 M_i 的第 k 个可见时间窗口，且由卫星 R_j 对其进行成像， $k \in \{1, \dots, N_{i,j}\}$ ， $N_{i,j}$ 表示任务 T_i 在整个仿真周期内所有资源的可见时间窗口数，而任务分配的执行时间只是选取该区间内满足成像时长约束的一小部分，则对于问题调度方案输出，可描述如式 2-1。

$$RESULT = \begin{cases} M_0, [tws_0, twe_0] \\ M_1, [tws_1, twe_1] \\ \dots \\ M_n, [tws_n, twe_n] \end{cases} \quad (2-1)$$

其中 $M_i \in M, [tws_i, twe_i] \subset [Beg_{i,j}^k, End_{i,j}^k]$

2.2.2 优化变量

优化变量包括分配任务的执行开始时间 t_i 和是否调度该任务 $x_{i,j}^k$ ，数学符号表示如下。

$$\begin{aligned} \forall M_i \in M, R_j \in R(M_i), k \in 1, \dots, N_{i,j}: \\ x_{i,j}^k \in 0,1, t_i \in R_0^+ \end{aligned} \quad (2-2)$$

2.2.3 优化目标

优化目标即在仿真时间内尽可能地使调度效益值最大，其中 w_i 为分配该任务所得效益值， $x_{i,j}^k$ 即调度该任务或不调度该任务。用数学符号表示如下。

$$\max \sum_{M_i \in M} \sum_{R_j \in R(M_i)} \sum_{k \in 1,2,\dots,N_{i,j}} w_i \cdot x_{i,j}^k \quad (2-3)$$

2.2.4 约束条件

从任务执行上来说，任务 M_i 最多只能被一个卫星 k 在某个时间段内调度一次。
用数学符号表示如下。

$\forall M_i \in M$:

$$\sum_{R_j \in R(M_i)} \sum_{k \in 1,2,\dots,N_{i,j}} x_{i,j}^k \leq 1 \quad (2-4)$$

从观测时间窗口上来说，分配任务的执行开始时间 t_i 在观测时间窗口范围内且观测时间窗口长度大于任务执行所需时间。用数学符号表示如下。

$$\text{若 } x_{i,j}^k = 1, \text{ 则 } Beg_{i,j}^k \leq t_i \leq End_{i,j}^k - D_{i,j} \quad (2-5)$$

其中 $D_{i,j}$ 为由卫星 j 成像的任务 M_i 所需执行时间。

从任务间转换时长来说，由卫星 j 在第 k 个时间窗口进行成像的两个任务 M_i 和 $M_{i'}$ ，需要满足后一个任务开始时间减去前一个任务结束时间的时间差大于等于卫星任务间转换时长。用数学符号表示如下。

$$\text{如果 } x_{i,j}^k = 1, x_{i',j}^{k'} = 1, \text{ 且有 } t_i < t_{i'}, \text{ 则 } t_{i'} - t_i \geq trans_k + D_{i,j} \quad (2-6)$$

其中 $trans_k$ 为卫星 k 任务间最小转换时长。

2.3 问题难点分析

2.3.1 卫星时间窗口难点分析

计算卫星对目标的可见时间窗口的难点在于如何判断目标点是否在给定的由二十个点（二十一个点首尾相接）围成的曲面多边形中。有两种思路，一种是利用曲面多边形求出圆的半径和圆心，再利用欧式距离判断点是否在卫星圆中；另一种是从图形的角度出发，利用射线法判断点是否在曲面多边形中。对于前者来说，可以通过读取圆上的三个点来求三角形的外接圆，以得到圆的半径和圆心，由于给出的二十一个点均匀分布在圆上，还可以分别读取第一个点和第十一个点，得到一条直径，从而求得圆的圆心和半径，提高计算效率。

2.3.2 卫星对区域瞬时覆盖率和累积覆盖率计算难点分析

区域覆盖率的计算采用了一种分治的思想，将面积的计算转换为网格的累加，难点在于在保证运行时间可接受的情况下提高精度。如何计算区域累积覆盖率也是难点之一，累积覆盖率不是每秒覆盖率的值单纯的累加，从几何层面上来理解，要考虑前一状态的覆盖矩形与当前状态覆盖矩形的并集。考虑为初始网格加入状态位，对有效网格进状态更改，计算已被覆盖过的矩形面积，用来近似累积覆盖率。在网格个数足够多、面积足够小的情况下，可以认为每一秒已被覆盖过的矩形累加面积除以总面积等于当前的累积覆盖率。同理在保证程序可行性的情况下，可以通过增加初始网格个数提高累积覆盖率的精度。

2.3.3 卫星星座对观测任务调度规划难点分析

卫星星座对观测任务进行调度是一个线性优化问题，在多个约束的条件下，对于具有不同特点的五组测试算例，采取何种策略能够增大效益值是问题的关键。一方面，需要考虑混合资源和时间窗口分配的复杂调度问题，以及多资源与观测时间窗口选择。对于一个任务，分配哪颗卫星进行分配、分配观测时间的时机都是决策的一部分；另一方面，决策时还要考虑多任务对有限资源争用冲突的消除。为一个任务分配的观测时间应该尽可能少的与其他任务可观测时间窗口发生冲突。

3 算法设计与实现

3.1 算法主要思想

3.1.1 读取卫星文件算法

对每一秒的多边形，取第一个点和第十一个点作为圆的直径，结合三角函数公式将经纬度坐标转换为三维坐标，利用欧式距离计算卫星圆的半径，循环读取。

3.1.2 合并时间窗口算法

该算法主要将输入的有效时间集合合并为卫星对目标观测的时间窗口区间，将有效时间视作长度为 1 的时间窗口，对相邻时间窗口进行区间合并即可。

3.1.3 计算二重时间窗口

对一个目标的 n 个时间窗口，设置两个指针 i 、 j ，指针 j 遍历除 i 外的所有时间窗口，若 i 、 j 指向时间窗口交集不为空，将二重覆盖时间窗口加入列表，直到 i 指向时间窗口 $n-1$ 时停止。

3.1.4 计算时间间隙算法

考虑到多卫星对同一目标的时间窗口会有交集，即二重或多重覆盖时间窗口，在求时间间隙时，需要对目标的时间窗口求并集，再对整个仿真时间求补集即得到时间间隙。

3.1.5 计算瞬时覆盖率和累积覆盖率算法

先将区域矩形以递归的形式划分成 n 个网格并加入队列中，一次递归将一个矩形划分成四个小矩形；然后遍历该网格队列。对于一个矩形，计算每个卫星圆对该矩形四个点的覆盖情况，记录覆盖点最大值，若覆盖点为 4，则表示全覆盖，将矩形标绿，状态位置 1；若覆盖点为 0，矩形标红；若覆盖点在 $[1, 3]$ 之间，矩形标黄，将其加入不确定队列中，等待分治思想细化矩形的计算。遍历完毕后，对不确定队列中的矩形进行划分，划分后再判断直到不确定矩形面积小于总面积的 0.1% 时停止。计算结束后对初始网格矩形中状态位为 1 的矩形进行面积计算用于近似累积覆盖率。

3.1.6 贪心分配任务算法

对于要分配的任务进行基于单位执行时间效益值的排序。卫星分配顺序为用户从键盘输入卫星序号顺序。对于观测卫星，遍历所有任务，优先分配优先值高即单位时间效益值大的未分配任务，选择满足执行时间的、开始时间最早的未分配时间段。

3.2 核心算法设计

3.2.1 读取卫星文件算法思想

对每二十一个点，读取第一个点和第十一个点的经度和维度，利用公式 3-1~3 将经纬度坐标转为三维坐标。其中 R 为地球半径， $latitude$ 为维度坐标， $longitude$ 为经度坐标，经纬度坐标均转为弧度制。

$$x = R \cdot \cos(latitude) \cdot \cos(longitude) \quad (3-1)$$

$$y = R \cdot \cos(latitude) \cdot \sin(longitude) \quad (3-2)$$

$$z = R \cdot \sin(latitude) \quad (3-3)$$

利用欧式距离计算两点间的距离， $d/2$ 即卫星圆的半径。

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2} \quad (3-4)$$

两点的距离即直径长度，两点的中点即圆心。将半径、卫星圆对应的时间、圆心的经纬度和三维坐标作为参数传入卫星类中，创建卫星类的一个实例，循环读取即可。

3.2.2 合并时间窗口算法思想

输入为自定义类的目标信息列表和卫星信息列表，输出为可观测时间窗口区间列表。

利用式 3-4 计算目标点与卫星圆圆心的欧式距离，判断与半径的关系如式 3-5。

$$\begin{cases} d < r, \text{目标点在圆内} \\ d \geq r, \text{目标点不在圆内} \end{cases} \quad (3-5)$$

对于在圆内的目标点，创建一个有效时间实例，包括可观测的时间、可观测的卫星序号和被观测的目标点。将有效时间以目标点名称为键存入字典中，再将相同键的有效时间按卫星序号分类存入列表中。形成目标点→卫星序号→有效时间三级索引，根据卫星对目标点的有效时间列表进行时间窗口的合并。合并时间窗口时先将时间按

大小排序，将初始有效时间看作长度为 1 的时间窗口，简单判断时间是否连续，是则递增，否则将区间加入时间窗口。最后返回可观测时间窗口区间。为得到二重覆盖时间窗口，对时间窗口求交集，将得到的二重覆盖时间窗口存入以二元组[开始时间，结束时间]为值的列表中；为得到时间间隙，对时间窗口先求并集再求补集，将得到的时间间隙以二元组[开始时间，结束时间]为值的列表中。

3.2.3 计算覆盖率算法思想

输入为自定义类的卫星信息、范围面积和已划分为 n 个网格矩形的队列，输出为时间、覆盖率和一系列矩形信息（用于可视化）。由分析可得划分网格必须满足面积小于卫星圆的面积，在保证运行时间可接收的情况下，本报告将网格划分为 512 个。利用球面微分的方法求矩形面积，见式 3-6。

$$Area = R * R * (\cos(latitude_1) - \cos(latitude_2)) * (lotitude_2 - lotitude_1) \quad (3-6)$$

设置变量瞬时覆盖率、累计覆盖率、不确定面积。设计两个队列，一个存储初始网格划分的矩阵，另一个用于存储不确定矩形。从队列中取出一个矩形，根据卫星圆对覆盖点个数的最大值判断矩形与圆的位置关系，如式 3-7。

$$\begin{cases} \text{覆盖点个数} = 4, \text{矩形在圆内} \\ 0 < \text{覆盖点个数} < 4, \text{矩形部分在圆内} \\ \text{覆盖点个数} = 0, \text{矩形不在圆内} \end{cases} \quad (3-7)$$

对在圆内的矩形，标记“绿色”，累加瞬时覆盖率，累减瞬时不确定面积；对不在圆内的矩形，标记“红色”，累减瞬时不确定面积；对部分在圆内的矩形，加入不确定队列或细分为四个小矩形再加入不确定队列。重复上面的步骤直到不确定矩形面积小于总面积的 0.1%。

在测试中发现，这种临界条件所需要的运行时间过长，考虑加入临界条件为不确定矩形面积小于总面积的 0.05% 时，不再进入队列，队列为空时，覆盖率计算结束。将每一秒的覆盖率和累积覆盖率写入文件并绘制曲线图。

3.2.4 贪婪任务调度算法思想

输入为以卫星序号为键的有效时间窗口，输出为任务调度序列和总效益值。将目标按照单位时间的效益值即效益值/任务时长进行排序。对每一个时间窗口，判断能否

找到一个可行的观测任务开始时间，该任务满足单位时间效益值尽可能得大。能则返回任务开始时间，否则认为无法分配资源。对于能够分配资源的任务，将任务状态设为 1，防止重复分配资源。由于相邻任务间存在卫星转换时间，分析易得卫星转换应在任务结束后立刻执行，由此可将执行观测任务时间和卫星转换时间段加入已分配时间列表。如何找到一个任务的开始时间，我们可以先计算[开始时间,结束时间]与已分配时间列表的补集，找到未分配的时间段，再计算和时间窗口的交集即可得到可观测的、未分配 的时间段，若该时间段长度满足观测任务所需时间加卫星转换所需时间则为可行的观测任务分配时间。简单地按照卫星序号的大小进行分配任务直到所有时间窗口都被遍历。

3.3 数据结构设计

3.3.1 目标类

```
class Target:
    self.name = name # 地名
    self.longitude = lo # 经度
    self.latitude = la # 纬度
```

3.3.2 卫星类

```
class Satellite:
    self.time = time # 卫星圆对应时间
    self.center_y = center_y # 圆心信息
    self.center_x = center_x
    self.center_z = center_z
    self.longitude = longitude
    self.latitude = latitude
    self.radius = radius # 半径
    self.num = num # 卫星序号
```

3.3.3 有效时间类

```
class Valid:

    self.time = time//时间

    self.satellite_num = satellite_num//卫星序号

    self.target = target//目标点名称
```

3.3.4 点类

```
class Point:

    self.latitude = la # 纬度

    self.longitude = lo # 经度

    self.x, self.y, self.z = ReadInfo.global_lonlat_topos(la, lo)# 经
    纬度坐标转三维坐标
```

3.3.5 四边形类

```
class Rectangle:

    """

    各点对应位置

    rectangle:{

                [left0,right0],

                [left1,right1]

            }

    """

    self.left0 = left0 # 左上角点

    self.left1 = left1 # 左下角点

    self.right0 = right0 # 右上角点

    self.right1 = right1 # 右下角点

    self.color = None # 多边形颜色

    self.area = area # 多边形面积

    self.state = 0 # # 多边形状态位
```

3.3.6 任务类

```
class Task:

    self.name = name

    self.longitude = lo # 经度

    self.latitude = la # 纬度

    self.x, self.y, self.z = self.lonla_topos() # 三维坐标

    self.time = time # 观测所需时间

    self.score = score # 观测效益值

    self.state = 0 # 状态值, 0--未安排该任务, 1--已安排该任务
```

3.4 算法模块设计

3.4.1 读文件模块设计

读文件模块存放在“ReadInfo.py”中, 包括读取目标文件模块和读取卫星文件模块。

3.4.2 时间窗口模块设计

时间窗口模块存放在“Problem_1.py”中, 包括读文件模块、计算有效时间模块、合并时间窗口模块、计算二重覆盖时间窗口模块、计算时间间隙模块和输出到文件模块。

3.4.3 覆盖率模块设计

覆盖率模块存放在“Problem_2.py”中, 包括读取卫星文件信息模块、初始化网格模块、计算瞬时覆盖率模块、计算累积覆盖率模块、可视化界面模块。其中计算瞬时覆盖率模块还包括了判断矩形与圆的位置关系模块、细分矩形模块。可视化界面模块包括 GUI 图形用户界面模块, 输出到文件模块和折线图绘制模块。

3.4.4 贪婪任务调度模块设计

贪婪任务调度模块存放在“Problem_3.py”中, 包括从键盘输入仿真信息模块、读取目标文件信息模块、读取卫星文件信息模块、计算时间窗口模块、贪婪调度算法模块和可视化模块。贪婪调度算法模块又包括查找可分配资源模块。

3.4.5 算法模块耦合关系

算法模块耦合关系如图 3-1、3-2、3-3 所示。

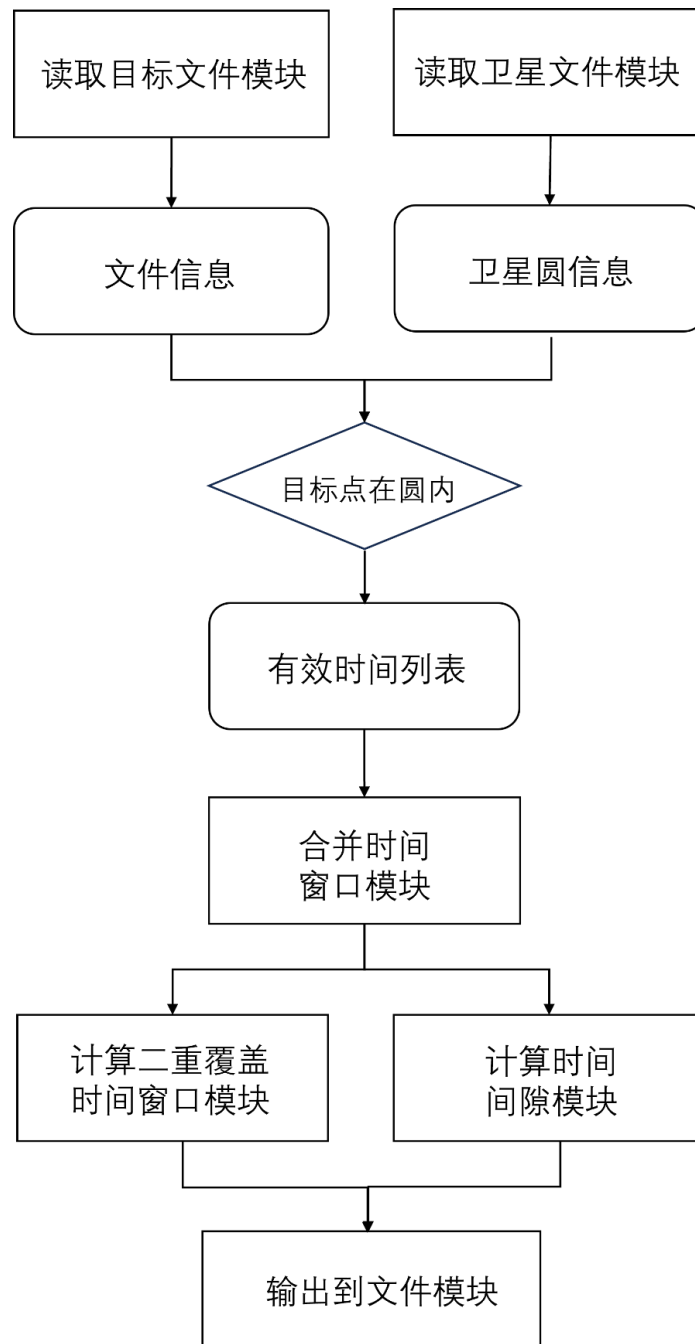


图 3-1 时间窗口模块

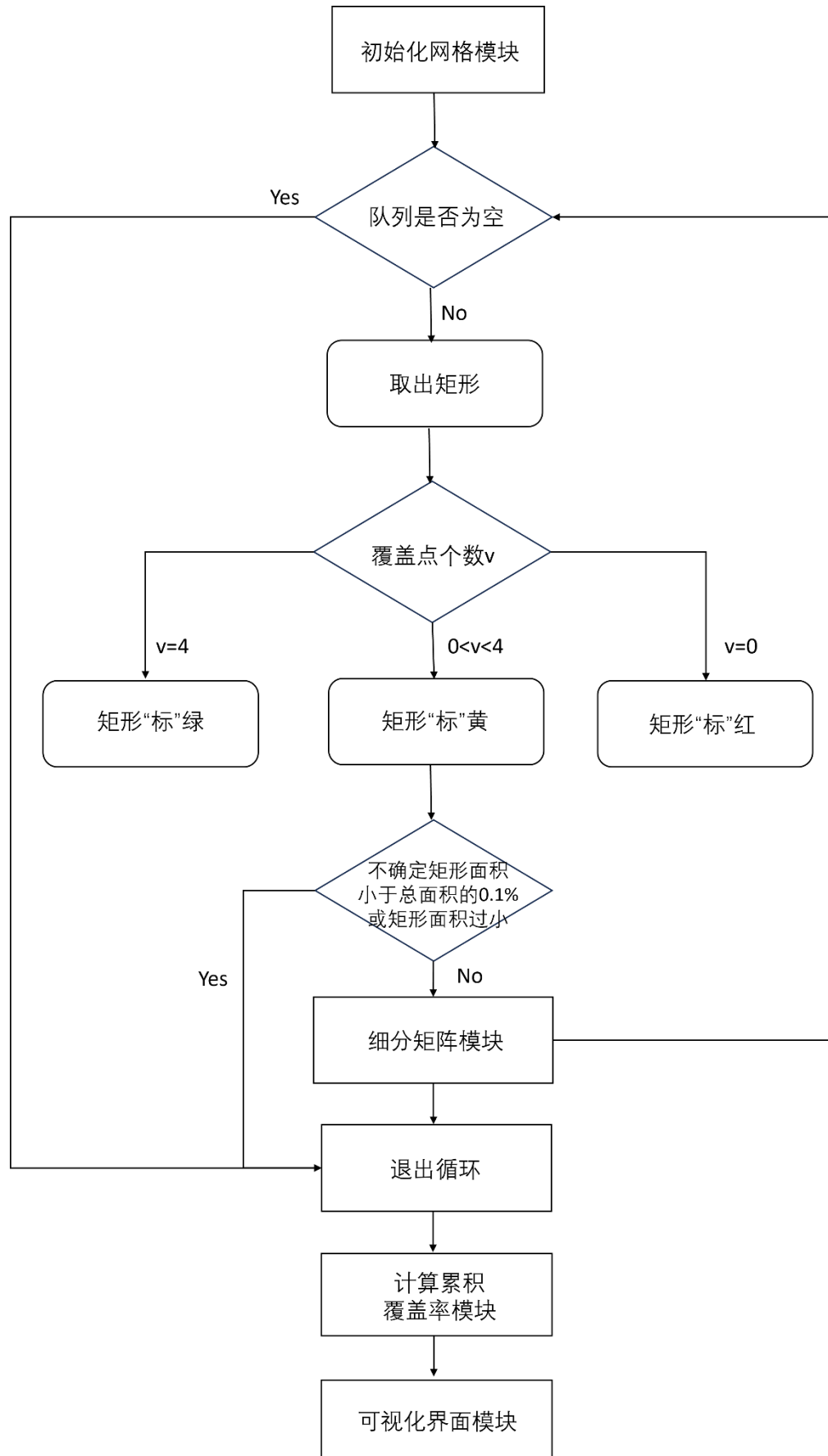


图 3-2 区域覆盖率计算模块

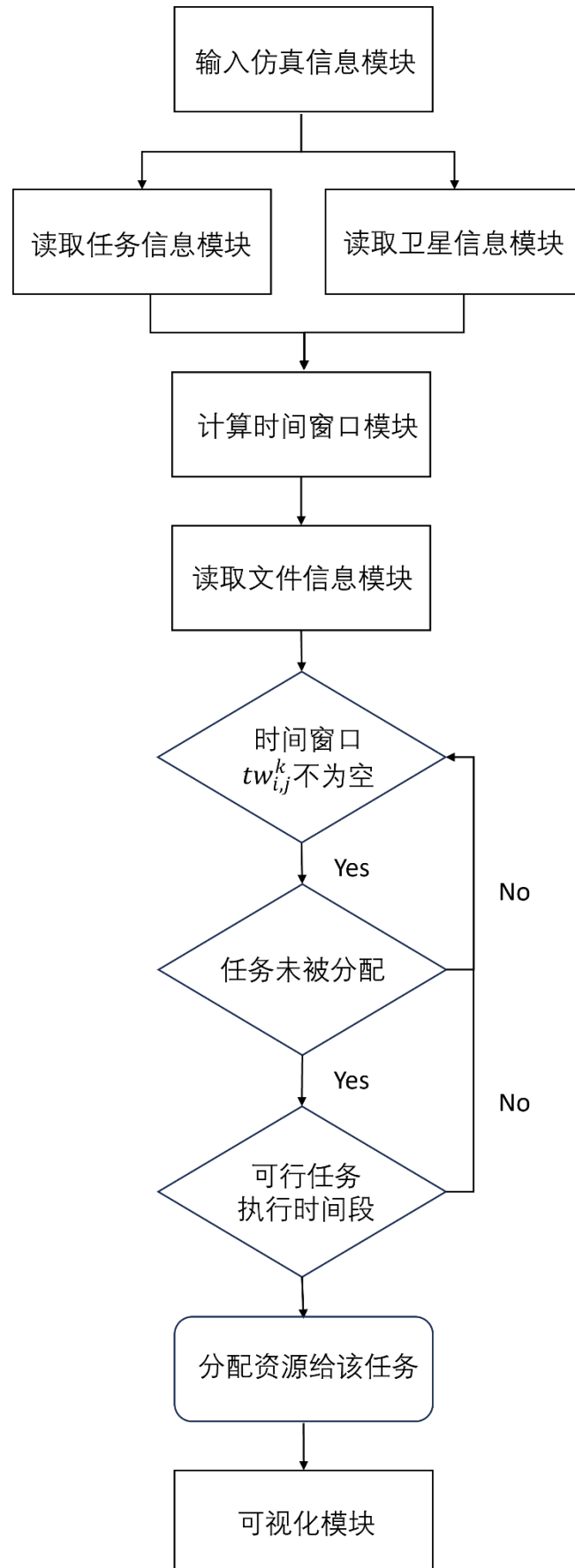


图 3-3 贪婪任务调度模块

4 结果测试

4.1 运行环境描述

操作系统: Windows 11

编译器: Pycharm2023.3.4

语言: Python

配置环境: requirement.txt, 包含六个库, “math”、“time”、“matplotlib.ticker”
“os”、“matplotlib.pyplot”、“tkinter”。

4.2 测试数据设计

4.2.1 时间窗口测试数据设计

计算时间窗口的测试数据可以分为两类, 一类是目标数据, 以[名称, 经度, 纬度]的三元组给出, 存放在“target.txt”中, 还有一类是卫星数据, 以日期、24 小时制的时间和 21 个[经度, 纬度]的形式给出, 每个卫星文件对应二十四小时的 86400 秒和第二天的 1 秒总共 86401 秒, 共有九颗卫星。

4.2.2 卫星对区域覆盖率数据设计

给定曲面多边形区域为一个经纬度矩形范围。经度区间为 75°E - 135°E , 纬度区间为 0°N - 55°N 。卫星数据与题目一中相同。

4.2.3 卫星对目标点任务调度数据设计

给定五组不同的测试案例, 测试案例间有时间、任务个数和可调度卫星上的区别。任务以四元组[目标点名称, 经度, 纬度, 任务时间, 效益值]的形式保存在 txt 文件中, 卫星数据和题目一中相同, 为了减少计算时间提高程序运行效率, 将所有卫星圆对应的圆心经度和纬度以三元组[时间, 经度, 纬度]的形式存放在 Trace 文件夹中。

4.3 测试结果分析

4.3.1 时间窗口输出分析

时间窗口输出为 24 个 txt 文件，对应 24 个目标的时间窗口及二重时间窗口、时间间隙、最大时间间隙和平均时间间隙。计算得到时间窗口误差在 2^3 秒左右，在误差范围内。

4.3.2 覆盖率输出分析

覆盖率输出包含三个部分：文本文件，GUI 界面和曲线图。文本文件中存储了覆盖率不为 0 的时间和它们相应的累积覆盖率以及最大覆盖率及其对应时间，文本文件中，最初覆盖率较低，如 0.04% 时，累积覆盖率为 0，分析原因为初始网格面积较大，还没有可以修改状态的初始网格导致的误差。由于累积覆盖率的计算依赖于初始网格划分大小，若初始网格划分较大，则累积覆盖率变化也较大，反之，累积覆盖率精度提升。瞬时覆盖率误差计算较大，但还未能找到合理的解释。对于 GUI 界面，开始/暂停按钮可让矩形以 60 秒为 1 个单位显示，最初设置为 1 秒，但展示时间过长；可从文本框输入时间，时间单位为秒，需要点击右部的上箭头或下箭头进行展示；也可以通过滑动滑块来查看范围覆盖情况，缺点是反应时间较长，分析原因为加入集合的矩形个数过多且可能存在冗余，导致绘制时间长，反应到可视化界面上即存在卡顿。GUI 界面中绘制的矩形以经度作为横坐标，纬度作为纵坐标简单表示，这种表示方法会导致绘制矩形构成圆的误差，边界区域圆由于形变会成为椭圆。

4.3.3 卫星观测任务调度输出分析

卫星观测任务调度输出包含 txt 文件和甘特图。不同目标文件生成不同的 txt 对应五个不同的测试案例。输出总效益值总体小于参考贪心解的效益值，测试时发现卫星顺序也会导致分配任务的变化，分析认为当前贪心策略考虑要素过少，无法充分分配观测任务。从甘特图中也可以看出，先分配的卫星调度任务较多，后分配的卫星调度任务较少。需要注意的是，甘特图重复的部分表示由其他卫星分配。

4.4 测试结果展示

4.4.1 时间窗口输出结果展示

时间窗口输出以目标 Abidjan 为例, 如下:

```
Target: Abidjan
1号卫星
2号卫星
Valid Time: ('2024/1/1 17:59:18', '2024/1/1 18:03:22'), Sum:244

3号卫星
Valid Time: ('2024/1/1 05:11:42', '2024/1/1 05:16:11'), Sum:269
Valid Time: ('2024/1/1 17:20:56', '2024/1/1 17:24:12'), Sum:196

4号卫星
Valid Time: ('2024/1/1 01:38:42', '2024/1/1 01:43:05'), Sum:263
Valid Time: ('2024/1/1 13:42:09', '2024/1/1 13:45:05'), Sum:176

5号卫星
Valid Time: ('2024/1/1 13:02:45', '2024/1/1 13:06:57'), Sum:252

6号卫星
Valid Time: ('2024/1/1 02:18:35', '2024/1/1 02:20:46'), Sum:131

7号卫星
Valid Time: ('2024/1/1 20:53:54', '2024/1/1 20:57:31'), Sum:217

8号卫星
Valid Time: ('2024/1/1 10:08:49', '2024/1/1 10:12:14'), Sum:205

9号卫星
Valid Time: ('2024/1/1 09:29:46', '2024/1/1 09:33:45'), Sum:239
Valid Time: ('2024/1/1 21:32:34', '2024/1/1 21:36:23'), Sum:229
```

图 4-1 时间窗口输出到文本文件

```
Target: Abidjan
Interval: ('2024/1/1 00:00:00', '2024/1/1 01:38:41')
Interval: ('2024/1/1 01:43:06', '2024/1/1 02:18:34')
Interval: ('2024/1/1 02:20:47', '2024/1/1 05:11:41')
Interval: ('2024/1/1 05:16:12', '2024/1/1 09:29:45')
Interval: ('2024/1/1 09:33:46', '2024/1/1 10:08:48')
Interval: ('2024/1/1 10:12:15', '2024/1/1 13:02:44')
Interval: ('2024/1/1 13:06:58', '2024/1/1 13:42:08')
Interval: ('2024/1/1 13:45:06', '2024/1/1 17:20:55')
Interval: ('2024/1/1 17:24:13', '2024/1/1 17:59:17')
Interval: ('2024/1/1 18:03:23', '2024/1/1 20:53:53')
Interval: ('2024/1/1 20:57:32', '2024/1/1 21:32:33')
Interval: ('2024/1/1 21:36:24', '2024/1/2 00:00:00')
最大时间空隙为:('2024/1/1 05:16:12', '2024/1/1 09:29:45'), 最大时间空隙大小为: 15213, 平均时间空隙大小为:6996.416666666667
```

图 4-1 时间间隙输出到文本文件

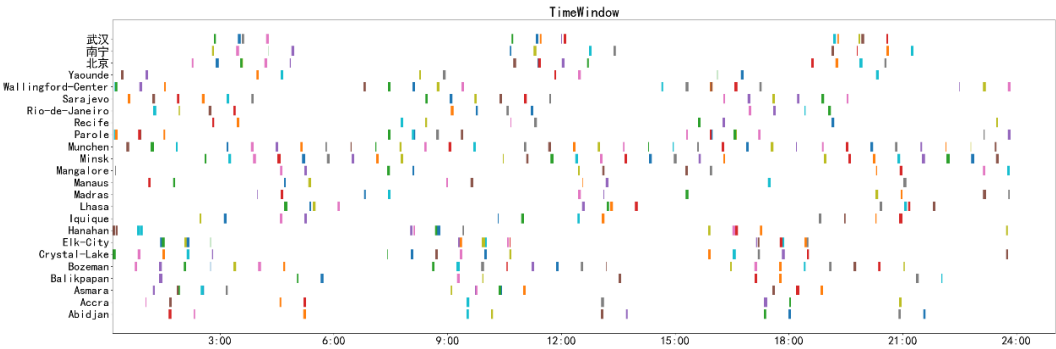


图 4-3 时间窗口甘特图

4. 4. 2 覆盖率输出结果展示

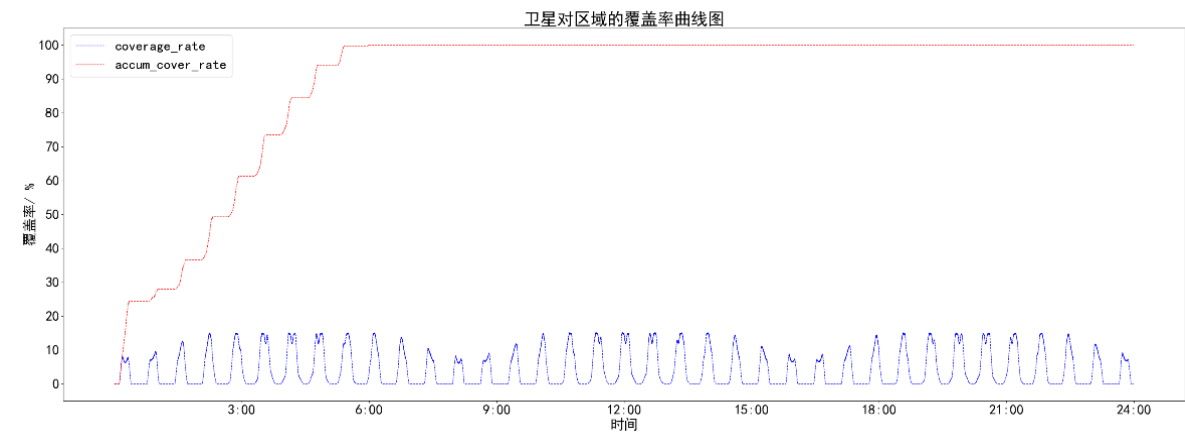


图 4-4 覆盖率曲线图（红：累积覆盖率，蓝：瞬时覆盖率）

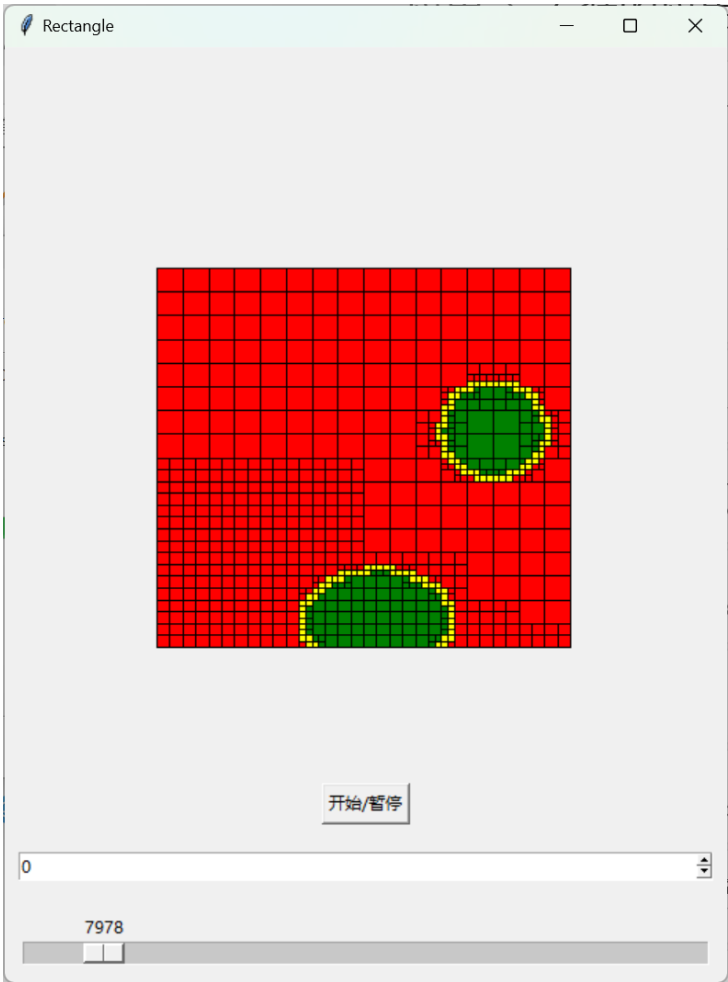


图 4-5 GUI 界面

任务：福州 任务时间：16.0 收益值：12.0
分配卫星序号：6 分配时间片：(41322.0, 41363.0)

任务：杭州 任务时间：16.0 收益值：12.0
分配卫星序号：6 分配时间片：(41170, 41211.0)

任务：西安 任务时间：16.0 收益值：12.0
分配卫星序号：6 分配时间片：(41045, 41086.0)

任务：青岛 任务时间：12.0 收益值：8.0
分配卫星序号：6 分配时间片：(41212.0, 41249.0)

任务：武汉 任务时间：9.0 收益值：6.0
分配卫星序号：6 分配时间片：(41250.0, 41284.0)

得分:262.0

图 4-8 输出到 txt 文件部分截图

5 分析总结

5.1 课设总结

本次卫星对地覆盖计算及任务规划实习紧密围绕卫星为地面目标提供服务的实际需求展开，深入探讨了贴近生活实际的应用问题。通过这次实习，我不仅对如何有效解决实际问题有了更为清晰的认识，而且深刻体会到了将实际问题抽象成数学模型的重要性。这种从具体到抽象、再从抽象到具体的思维过程，锻炼了我的逻辑思维和问题解决能力。本次实习题目是对实际问题的合理简化，从点、区域范围、以及时空交织这三个关键层面出发，进行了系统性的分析和建模。第一题侧重对圆和点位置关系的处理以及对区间的处理；第二题采用了分治法的思想，将难以求解的覆盖率转换为易求得网格矩形面积的累加。第三题是多目标规划问题，需要提出尽可能好的资源调度方案，不同策略适用得测试案例也不尽相同。

本次实习也是我第一次使用 `python` 编写代码，通过编写代码和利用开源图形界面库 `tkinter`，熟悉了 `python` 常用的数据结构、语法等，深刻体会到 `python` 与 `C++` 程序语言之间的区别和联系。在编写程序、不断调试、修改细节、细化想法和结构的过程中，实习题目的思路与想法不再只停留在题目中，而是一步一步地实践在编译中，尽管结果不算尽人意，其中收获的成就感和对实现时对细节的把握也令我心潮澎湃。最后十分感谢在实习过程中和我沟通交流思路的同学们和耐心指导的陈老师。

5.2 特色亮点

5.2.1 能够将输出结果转化为多种可视化信息，包括甘特图、GUI 界面、曲线图等，从而显著提高了结果的可读性和易用性。

5.2.2 通过将大任务有效地划分成小任务，并将其封装到函数中，显著提高了代码的复用性和可读性。

5.3 后期改进之处

5.3.1 改进数据结构，在写报告时发现第一题中的有效时间类冗余。

5.3.2 对于瞬时覆盖率计算误差较大的情况，需要分析出误差产生的原因，再细化代码中的细节。

5.3.3 将任务调度的测试案例写入文本文件中，编写读取文件函数，取消从键盘输入信息，提高便携性。

5.3.4 增加任务调度策略，在排序了单位时间的效益值的基础上，可以对多个可选择时间窗口上采用不同的策略，可以考虑的调度策略有空闲时间长/短的先分配；在不同卫星上也可以考虑不同的策略，如先分配两卫星可观测到目标点的交集的并集。也可以考虑改变排序策略，多次实验并对比不同调度策略分配得到的结果以及分析不同策略适用的测试算例特点。

6 附录

为提高代码可读性，删除可视化输出相关代码，仅展示算法结构代码和核心源码。

6.1 计算时间窗口模块

6.1.1 合并时间窗口函数

```
def merge_datetime(categories):  
    """  
    :param categories: 以目标为键，卫星序号和时间为值的类  
    :return: 输出 23 个文件，包含了 23 个目标在 9 个卫星下的可见时间窗口、二  
    重时间窗口和时间间隙  
    """  
    target_num_interval = defaultdict(list)  
    j = 0  
    for target, groups in categories.items():  
        num_intervals = [] # 还未转化成日期的有效观测时间（循环内对每个目  
        标有效）  
        times = {key: [] for key in range(1, 10)} # 为字典类型，键为卫星  
        序号，值为覆盖时间列表  
        for group in groups:  
            times[group.satellite_num].append(group.time)# 按照卫星序号重  
            分类  
        # 进行区间合并  
        for i in range(1, 10):  
            if times[i]:  
                intervals = merge_time(times[i]) #区间合并  
                num_intervals.append(intervals)  
                for interval in intervals:  
                    # 日期转换  
                    datetime_start = to_date(interval[0])  
                    datetime_end = to_date(interval[1])  
                    sum = interval[1] - interval[0]  
                    target_num_interval[target].append((interval[0],  
                    interval[1]))  
                j += 1  
    # 求二重覆盖时间，函数返回二重覆盖时间窗口
```

```

intersection_interval = intersection_intervals(num_intervals)
for interval in intersection_interval:
    if interval:
        for time in interval:
            time_start = to_date(time[0])
            time_end = to_date(time[1])
            res_sum = time[1] - time[0] #二重覆盖时间长度
        calc_time_intervals(target_num_interval, folder)

```

6.1.2 计算有效时间函数

```

def valid_timewindow():
    """
    :return: 未合并的有效时间窗口
    """
    # 对每个目标，每个卫星圆
    categories = defaultdict(list)
    for target in target_list:
        categories[target.name] = []
        # 经纬度坐标到三维坐标的转换
        target_x, target_y, target_z =
ReadInfo.global_lonlat_topos(target.latitude, target.longitude)
        # 欧式距离计算卫星圆与目标点的位置关系
        for i, satellite in enumerate(satellite_list):
            distance = math.sqrt((target_x - satellite.center_x) ** 2 +
(target_y - satellite.center_y) ** 2+ (target_z - satellite.center_z) **
2)
            if distance <= satellite.radius:
                valid = Valid(satellite.time, i // 86401 + 1, target.name)
                categories[target.name].append(valid)

```

6.1.3 计算二重覆盖时间窗口函数

```

def intersection_intervals(intervals):
    """
    :param intervals: 一个目标对应所有卫星星座的有效观测时间
    :return: 二重覆盖时间间隔
    """
    intersections = []
    for j in range(len(intervals)):
        for i in range(j + 1, len(intervals)):

```

```
        intersection_interval =  
        smaller_intersection_interval(intervals[i], intervals[j])  
        if intersection_interval:  
            intersections.append(intersection_interval)  
    return intersections
```

6.1.4 计算单个目标的二重覆盖时间窗口函数

```
def smaller_intersection_interval(intervals_A, intervals_B):  
    """  
    :param intervals_A: 卫星 A 对目标的时间窗口  
    :param intervals_B: 卫星 B 对目标的时间窗口  
    :return: 二重覆盖时间窗口  
    """  
  
    if not intervals_A or not intervals_B:  
        return None  
    # 升序排序，起始时间相同时结束时间小的在前  
    intervals_A.sort(key=lambda x: (x[0], -x[1]))  
    intervals_B.sort(key=lambda x: (x[0], -x[1]))  
    res = []  
    i = 0  
    j = 0  
    while i < len(intervals_A) and j < len(intervals_B):  
        a1, a2 = intervals_A[i][0], intervals_A[i][1]  
        b1, b2 = intervals_B[j][0], intervals_B[j][1]  
        if b2 >= a1 and a2 >= b1: # 时间窗口有交集  
            res.append([max(a1, b1), min(a2, b2)])  
        if b2 < a2:  
            j += 1  
        else:  
            i += 1  
    return res
```

6.1.5 合并时间窗口函数

```
def merge_time(times):  
    """  
    :param times: 一个卫星对一个目标的可见时间  
    :return: 可见时间窗口  
    """  
  
    merge = []
```

```
start = times[0]
startl = 0
end = start
for time in times[1:]:
    if time == end + 1: # 时间连续，更新时间窗口范围
        end = time
    else:
        if start == end:
            continue
        merge.append((start, end))
        start = time
        end = time
if start != end: # 最后一个时间窗口
    merge.append((start, end))
return merge
```

6.1.6 计算时间间隙函数

```
def calc_time_intervals(target_num_interval, folder):
    """
    :param target_num_interval: 目标点的所有时间窗口
    :return: 时间间隙
    """
    # 求并集，求补集
    result = {}
    result = defaultdict(list)
    for target in target_num_interval.keys():
        target_num_interval[target] = sorted(target_num_interval[target],
        key=lambda x: x[0])
        for interval in target_num_interval[target]:
            # result 中最后一个区间的右值>=新区间的左值，说明两个区间有重叠
            if result[target] and result[target][-1][1] >= interval[0]:
                # 将 result 中最后一个区间更新为合并之后的新区间
                new_end = max(result[target][-1][1], interval[1])
                s, e = result[target].pop()
                result[target].append((s, new_end)) # 合并
            else:
                result[target].append(interval)
    #对一天的仿真时间进行求补
```

```
end1 = 0
start2 = 0
end2 = 86400
i = 0
j = 0
bu_result = [] # 补集
for new_interval in result[target]:
    start1, end1 = new_interval
    if start1 > start2:
        if start2 == 0:
            bu_result.append((start2, start1 - 1))
        else:
            bu_result.append((start2 + 1, start1 - 1))
        start2 = end1
    if end1 + 1 < end2:
        bu_result.append((end1 + 1, end2))
# 找最大时间间隙
max_interval = 0
max_intervals = (0, 0)
accum_sum = 0
for interval in bu_result:
    start, end = interval
    sum = end - start
    accum_sum += sum
    start = to_date(start)
    end = to_date(end)
    if sum > max_interval:
        max_interval = sum
        max_intervals = (start, end)
```

6.2 计算覆盖率模块

6.2.1 瞬时覆盖率模块

```
def cal_coverage_rate(satellites, rectangle_area, queue):
    """
    :param satellites: 九个卫星圆信息
    :param rectangle_area: 总矩形信息
    :param queue: 矩形队列
    :return: 时间、覆盖率和有效矩形
    """
```

```
# 检查一个矩形的四个点是否都在观测范围内
valid_rect = []
new_queue = []
coverage_rate = 0
area0 = rectangle_area.area * 0.001
accum_yellow = rectangle_area.area
for m in range(0, len(queue)):
    rectangle = queue[m]
    i = 0
    for satellite in satellites:
        j = check_points_valid(rectangle, satellite)
        if j > i:
            i = j
        if i == 4:
            break
    if i == 4:
        temp=Rectangle(rectangle.left0,rectangle.right0,
rectangle.left1, rectangle.right1, rectangle.area)
        accum_yellow -= rectangle.area
        coverage_rate += rectangle.area
        temp.color = 'green'
        rectangle.state = 1
        valid_rect.append(temp)
    elif i == 0:
        temp=Rectangle(rectangle.left0,rectangle.right0,
rectangle.left1, rectangle.right1, rectangle.area)
        accum_yellow -= rectangle.area
        temp.color = 'red'
        valid_rect.append(temp)
    else:
        temp=Rectangle(rectangle.left0,rectangle.right0,
rectangle.left1, rectangle.right1, rectangle.area)
        # 不确定的网格面积之和与总面积之比小于 0.1%
        temp.color = 'yellow'
        valid_rect.append(temp)
        new_queue.append(temp)
    m += 1
# 对于不确定矩形队列中的元素
while new_queue:
```

```
    if accum_yellow < area0:
        break
    i = 0
    rect = new_queue[0]
    new_queue.remove(rect)
    for satellite in satellites:
        j = check_points_valid(rect, satellite)
        if j > i:
            i = j
        if i == 4:
            break
    if i == 4:
        #填充为绿色
        accum_yellow -= rect.area
        coverage_rate += rect.area
        rect.color = 'green'
        valid_rect.append(rect)
    elif i == 0:
        # 填充为红色
        accum_yellow -= rect.area
        rect.color = 'red'
        valid_rect.append(rect)
    else:
        # 填充为黄色，调用 split 函数分割矩形再入队列
        rect.color = 'yellow'
        valid_rect.append(rect)
        # 不确定的网格面积之和与总面积之比小于 0.1%
        if rect.area < 0.5 * area0:
            continue
        if rect and accum_yellow > area0:
            new_rectangle = split_rectangle_area(rect)
            new_queue.append(new_rectangle[0])
            new_queue.append(new_rectangle[1])
            new_queue.append(new_rectangle[2])
            new_queue.append(new_rectangle[3])
        else:
            break

# 返回
if coverage_rate:
```



```

        coverage_rate /= rectangle_area.area
        coverage_rate *= 100 # 百分率
        return satellites[1].time, coverage_rate, valid_rect
    else:
        return satellites[1].time, 0, valid_rect

```

6.2.2 判断矩形与卫星圆的关系

```

def check_points_valid(rectangle, satellite):
    """
    :param rectangle: 矩形信息
    :param satellite: 卫星圆信息
    :return: 覆盖点个数
    """
    i = 0
    if (cal_distance(rectangle.left0.lo, rectangle.left0.la,
                     satellite.longitude, satellite.latitude)
        <= satellite.radius):
        i += 1
    if (cal_distance(rectangle.left1.lo, rectangle.left0.la,
                     satellite.longitude, satellite.latitude)
        <= satellite.radius):
        i += 1
    if (cal_distance(rectangle.right0.lo, rectangle.right0.la,
                     satellite.longitude, satellite.latitude)
        <= satellite.radius):
        i += 1
    if (cal_distance(rectangle.right1.lo, rectangle.right1.la,
                     satellite.longitude, satellite.latitude)
        <= satellite.radius):
        i += 1
    return i

```

6.2.4 计算矩形面积

```

def calc_area(lo1, lo2, la1, la2):
    """
    :param lo1, lo2, la1, la2: 从小到大、从经度到纬度
    :return: 矩形面积
    """
    lo1, lo2, la1, la2 = map(math.radians,
                             [lo1, lo2, la1, la2])

```

```
return R * R * (math.cos(la1) - math.cos(la2)) * (lo2 - lo1)
```

6.2.5 细分矩形模块

```
def split_rectangle_area(rectangle_area):  
    """  
    :param rectangle_area: 要分割的矩形信息  
    :return: 细化后的四个矩形信息  
    """  
  
    lo1, lo2, la1, la2 = (rectangle_area.left0.lo,  
rectangle_area.right0.lo, rectangle_area.left1.la, rectangle_area.left0.la)  
  
    new_rectangle_point = []  
    # 四个中点和中心点坐标  
    new_point_center=Point(  
        (rectangle_area.left0.lo + rectangle_area.right0.lo) / 2,  
(rectangle_area.left0.la + rectangle_area.left1.la) / 2)  
    new_point_01 = Point(  
        (rectangle_area.left0.lo + rectangle_area.right0.lo) / 2,  
rectangle_area.left0.la)  
    new_point_02 = Point(rectangle_area.left0.lo,  
(rectangle_area.left0.la + rectangle_area.left1.la) / 2)  
    new_point_13 = Point(rectangle_area.right0.lo,  
(rectangle_area.right0.la + rectangle_area.right1.la) / 2)  
    new_point_23 = Point(  
        (rectangle_area.left1.lo + rectangle_area.right1.lo) / 2,  
rectangle_area.left1.la)  
    # 从左往右, 从上往下  
    # 1  
    area = calc_area(lo1, (lo1 + lo2) / 2, (la2 + la1) / 2, la2)  
    new_rectangle_point.append(Rectangle(rectangle_area.left0,  
new_point_01, new_point_02, new_point_center, area))  
    # 2  
    area = calc_area((lo1 + lo2) / 2, lo2, (la2 + la1) / 2, la2)  
    new_rectangle_point.append(Rectangle(new_point_01,  
rectangle_area.right0, new_point_center, new_point_13, area))  
    # 3  
    area = calc_area(lo1, (lo1 + lo2) / 2, la1, (la2 + la1) / 2)  
    new_rectangle_point.append(Rectangle(new_point_02, new_point_center,  
rectangle_area.left1, new_point_23, area))
```

```
# 4
area = calc_area((lo1 + lo2)/2, lo2, la1, (la2 + la1) / 2)
new_rectangle_point.append(Rectangle(new_point_center, new_point_l3,
new_point_23, rectangle_area.right1, area))
# 返回包含了四个矩形信息的列表
return new_rectangle_point
```

6.2.6 初始化队列模块

```
def init_queue(rectangle_area):
    """
    :param rectangle_area: 矩形区域信息
    :return: 将初始矩形分割成 512 个网格
    """
    queue = [rectangle_area]
    while len(queue) < 512:
        new_rectangle = split_rectangle_area(queue[0])
        queue.append(new_rectangle[0])
        queue.append(new_rectangle[1])
        queue.append(new_rectangle[2])
        queue.append(new_rectangle[3])
        queue.remove(queue[0])
    return queue
```

6.2.7 计算累积覆盖率模块

```
def calc_accum_rate(queue):
    """
    :param queue: 队列信息
    :return: 累积覆盖率
    """
    accum_coverage_rate = 0
    for rect in queue:
        if rect.state == 1:
            accum_coverage_rate += rect.area
    accum_coverage_rate /= rectangle_area.area
    accum_coverage_rate *= 100
    return accum_coverage_rate
```

6.2.8 主函数模块

```
if __name__ == '__main__':
    area = calc_area(75, 135, 0, 55)
    print(area)

    rectangle_area = Rectangle(Point(75, 55), Point(135, 55), Point(75, 0),
Point(135, 0), area)
    satellite_list = read_sat_info()
    valid_time_list = [] # 用于将覆盖率不为0的时间写入文件
    time_list = []
    # per_rect_list={key:[] for key in range(0,86400)}
    per_rect_list = []

    # 变量
    max_coverage_rate = 0
    per_accum_coverage_rate = []

    queue = init_queue(rectangle_area)
    for i in range(0, len(satellite_list)): # len(satellite_list)
        valid_time, coverage_rate, valid_rect= cal_coverage_rate(
satellites=satellite_list[i],
rectangle_area=rectangle_area, queue=queue)
        time_list.append((valid_time, coverage_rate))
        per_rect_list.append(valid_rect)
        accum_coverage_rate = 0
        if coverage_rate:
            if coverage_rate > max_coverage_rate:
                max_coverage_rate = coverage_rate
                max_time = valid_time
            valid_time_list.append((valid_time, coverage_rate,
accum_coverage_rate))
            accum_coverage_rate = calc_accum_rate(queue)
            per_accum_coverage_rate.append(accum_coverage_rate)
```

6.3 卫星调度任务模块

由于有效时间窗口求法与题目一类似，此处不再特意列出。

6.3.1 读文件信息模块

```
def read_target_info(target_info):
    """
```

```
:return: 任务信息
"""

directory_path = "D:\\Data\\TargetInfo\\"
i = 1
for filename in os.listdir(directory_path):
    if os.path.isfile(os.path.join(directory_path, filename)) and
filename.endswith(".txt"):
        file_path = os.path.join(directory_path, filename)
        with (open(file_path, 'r', encoding='utf-8') as f):
            for line in f:
                parts = line.split()
                if len(parts) == 5:
                    target_i = Task(parts[0], float(parts[1]),
float(parts[2]), float(parts[3]), float(parts[4]))
                    target_info[i].append(target_i)
                i += 1
return target_info
```

6.3.2 读卫星信息模块

```
def read_sat_info(sat_info):
    """
    :return: 卫星信息
    """

    directory_path = "D:\\Data\\Trace\\"
    # sat_info = {key: [] for key in range(1, 10)}
    i = 0
    for filename in os.listdir(directory_path):
        if os.path.isfile(os.path.join(directory_path, filename)):
            file_path = os.path.join(directory_path, filename)
            with (open(file_path, 'r', encoding='utf-8') as f):
                for line in f:
                    parts = line.split()
                    if len(parts) == 6 and (parts[0] == '1' or parts[0]
== '2'):
                        hours, minute, sec = parts[3].split(":")
                        sec, millisecond = sec.split('.')
                        total_seconds = int(hours) * 3600 + int(minute) *
60 + int(sec)

                        x, y, z = ReadInfo.global_lonlat_topos(
```

```
float(parts[4]), float(parts[5]))
        sat_i = Satellite(latitude=float(parts[4]),
longitud=longitude=float(parts[5]),center_x=x, center_y=y, center_z=z, num=i,
sta=int(total_seconds), radius=777.18) # 卫星半径来自题目一中求得的
        sat_info[i].append(sat_i)

        i += 1
    return sat_info
```

6.3.3 查找可分配资源

```
def check_busy(busy, task, valid_time, end, need):
    """
    :param busy: 已分配时间片
    :param task: 任务信息
    :param valid_time: 时间窗口
    :param end: 仿真结束时间
    :param need: 任务执行所需时间+卫星转换时长
    :return: 可分配任务执行时间段的开始时间
    """
    # 先求补集，再求交集
    valid_start = -1
    end1 = 0
    start2 = 0
    end2 = end
    i = 0
    j = 0
    bu_result = []
    busy = sorted(busy, key=lambda x: x[1][0])
    for task, busy_time in busy:
        start1, end1 = busy_time
        if start1 > start2:
            if start2 == 0:
                bu_result.append((start2, start1 - 1))
            else:
                bu_result.append((start2 + 1, start1 - 1))
            start2 = end1
    if end1 + 1 < end2:
        bu_result.append((end1 + 1, end2))
    for time in valid_time:
        # 无可用开始时间
```

```
    if time[1] - time[0] < task.time:
        continue
    if time[1] < time[0]:
        continue
    # 求 busy 和 (0, 86401) 的补集, 得到空闲时间片
    if not busy:
        valid_start = time[0]
        break
    else:
        busy = sorted(busy, key=lambda x: x[1])
    # 求空闲时间片和有效时间的交集, 限制时间长度
    start2 = time[0]
    end2 = time[1]
    for free_time in bu_result:
        start1 = free_time[0]
        end1 = free_time[1]
        if start1 < start2 and end1 - start2 > need:
            valid_start = start2
            break
        elif start1 > start2:
            if end1 < end2 and end1 - start1 > need:
                valid_start = start1
                break
            elif end1 > end2 and end2 - start1 > need:
                valid_start = start1
                break
    return valid_start
```

6.3.4 贪婪分配任务资源算法模块

```
def greedy_allocate_resource(end, valid_time_i, file_handle):
    """
    :param end: 仿真结束时间
    :param valid_time_i: 以卫星序号为键的时间窗口
    :return: 输出到文件和图片
    """
    profit = 0
    for key in valid_time_i:
        busy = []
        for task, valid_time in valid_time_i[key]:
            if task.state == 0:
```

```
        need = task.time + transform_smallest_time[key]
        valid_time_start = check_busy(busy, task, valid_time, end,
need) # 返回一个可行的任务调度开始时间
        if valid_time_start != -1:
            profit += task.score
            # busy 里是二元组 (目标相关信息, (开始时间, 结束时
间))
            busy.append((task, (valid_time_start, valid_time_start
+ need)))
            task.state = 1

    return forpaint
```